Case Study

RISC vs. CISC: Choosing a Processor for a Low-Power Embedded Device

Report

Course	Computer Organization and Architecture (25CST-238)
Student	Shriya
Roll number	25BAI10016
Programme	B.Tech CSE (AI), Chandigarh University
Submitted to	Ruchika Gupta

Contents

1. Introduction
2. Scenario and problem statement
3. Background: what an instruction set does and how performance is measured
4. RISC in detail
5. CISC in detail
6. Answers to the case study questions
7. Assignment (a): comparison table
8. Assignment (b): example instruction sequences
9. Pipelining and why it favours RISC
10. Power and energy: the core of the embedded decision
11. Assignment (c): processor recommendation
12. How blurred the line is today
13. Conclusion
14. Glossary
15. References

Abstract

An instruction set is the contract between software and hardware, and the way it is designed affects chip size, speed, cost and energy use. This report compares the two classic design styles, RISC and CISC, for one specific job: a small, battery-powered embedded device. It answers five case study questions, gives a twelve-parameter comparison table, works through instruction sequences for both styles, and explains the role of pipelining and power. The conclusion is that a RISC-style processor, for example an ARM Cortex-M or a RISC-V microcontroller, is the better fit, mainly because simpler decoding and uniform instructions reduce energy per task and make timing predictable. The report also states honestly where CISC wins and why the difference between the two styles is smaller today than it was in the 1980s.

1. Introduction

Every program, whether it is a phone app or the firmware in a fitness band, is eventually turned into a series of very small steps that the processor can carry out: add two numbers, fetch a value from memory, jump to another place in the program. The list of those steps, together with the way each step is written as bits, is called the **instruction set architecture** (ISA). It is the point where software ends and hardware begins.

Two broad philosophies have shaped ISAs for the last forty years. One says that the processor should offer a rich set of powerful instructions so that programs stay short. That is the CISC approach. The other says that the processor should offer only a few simple instructions and execute them extremely quickly, leaving the compiler to assemble bigger tasks out of small pieces. That is the RISC approach. Neither is simply right or wrong. Each makes a different trade between hardware complexity and software length, and the best choice depends on what the device is supposed to do.

In this case study the device is a low-power embedded one. Such devices run on small batteries, often for months or years, and usually do a narrow job such as reading a sensor, controlling a motor or sending a short radio message. The question is which style suits that job better, and the answer has to be justified, not just stated.

2. Scenario and Problem Statement

Scenario. A technology company develops two computing systems: one uses a RISC architecture and the other uses a CISC architecture. The team must select a suitable instruction-set design for a low-power embedded device.

Problem statement. Compare the two processor architectures and justify the preferred option for a given use case.

Use case chosen. A battery-powered embedded device such as a wireless sensor node, a wearable or a small controller. For this kind of product the priorities, in rough order, are:

- energy used per task, because that decides battery life;
- chip area and cost, because these devices are made in large numbers;
- predictable timing, because many of them must respond within a fixed deadline;
- adequate, not maximum, performance.

Deliverables. The assignment asks for (a) a comparison table with at least five parameters, (b) two example instructions or instruction sequences, and (c) a reasoned recommendation. This extended report provides all three and also answers the five questions in the case study.

3. Background: What an Instruction Set Does and How Performance Is Measured

An ISA defines the instructions available, the registers the programmer can see, the data types, the ways of naming a memory location (addressing modes), and the binary format of each instruction. Two processors that implement the same ISA can run the same programs, even if their insides are very different. This is why Intel and AMD can both make x86 chips and why many companies can make chips that run ARM code.

A standard way to reason about processor speed is the CPU performance equation:

```
CPU time  =  Instruction count  x  Cycles per instruction (CPI)  x  Clock cycle time
```

Each factor can be pushed by design. A CISC design tries to lower the **instruction count**, since one complex instruction replaces several simple ones. A RISC design tries to lower **CPI** and the **cycle time**, because simple instructions can finish in about one cycle and the logic is short enough to allow a fast clock. RISC usually pays with a higher instruction count, CISC usually pays with a higher CPI and a longer cycle. The RISC argument, first made clearly by Patterson and Ditzel in 1980, was that the second trade is the better one. The reasoning is that a compiler can generate simple instructions easily, while a lot of the complicated instructions in CISC machines were rarely used by compilers at all.

For an embedded device, a fourth quantity matters at least as much: **energy per task**. It depends on how many transistors switch during the work, how long the work takes, and what voltage and frequency the chip runs at. Section 10 returns to this.

4. RISC in Detail

Origins. The RISC idea came out of research projects in the early 1980s, notably the IBM 801, the Berkeley RISC project and the Stanford MIPS project. Researchers had measured real programs and found that compilers used only a small fraction of the available instructions most of the time. Making the common ones fast looked more promising than supporting rare, complicated ones.

Main design principles.

- **Load/store architecture.** Only LOAD and STORE instructions touch memory. Arithmetic and logic instructions work on registers only.
- **Fixed-length instructions.** Typically 32 bits, with fields in the same positions, so decoding is simple.
- **Few addressing modes.** Register, immediate and base-plus-offset cover most needs.
- **Large register file.** Having 16 to 32 general-purpose registers lets the compiler keep values on the chip instead of going to memory.
- **Hardwired control.** The control unit is built from logic gates rather than microcode, which is faster and smaller for simple instruction sets.
- **Designed for pipelining.** Uniform instructions and uniform time per stage keep the pipeline full.

Example of a regular format. The 32-bit R-type instruction of RISC-V shows how regular a RISC encoding is. Every field sits in the same place in every instruction of this type:

Field	funct7	rs2	rs1	funct3	rd	opcode
Bits	7	5	5	3	5	7
Meaning	Operation variant	Source reg 2	Source reg 1	Operation group	Destination reg	Instruction class

Figure 1: Layout of a RISC-V R-type instruction (7 + 5 + 5 + 3 + 5 + 7 = 32 bits). The decoder only has to slice the word at fixed positions.

Where RISC is used. ARM cores power most smartphones and nearly all microcontrollers in the Cortex-M family. RISC-V is an open ISA that is growing in embedded and academic use. MIPS has long been used in networking equipment and teaching. The common thread is that these markets care about efficiency.

5. CISC in Detail

Origins. CISC designs date from the era when memory was expensive and slow and when most programs were written in assembly language. In that setting it made sense to pack a lot of work into each instruction: programs became shorter, fewer bytes had to be fetched, and the hardware could offer instructions that looked a bit like statements of a high-level language. Microcode made this practical, because a complicated instruction could be implemented as a small internal program stored in a ROM. The IBM System/360, the DEC VAX and the Motorola 68000 all followed this path, and so did Intel's 8086 in 1978, the ancestor of today's x86.

Main characteristics.

- a large number of instructions, some very specialised (string operations, loop instructions, bit-field instructions);
- variable-length encoding, so common instructions can be short and rare ones long;
- many addressing modes, with memory operands allowed in most instructions;
- a small number of general-purpose registers in older designs;
- microprogrammed control for the complex instructions;
- high code density, which saves memory and instruction-fetch bandwidth.

The cost of variable length. x86 is a good example. An instruction is built from optional parts, so its length is not known until some of its bytes have been read:

Part of an x86 instruction	Size (bytes)
Legacy prefixes	0 to 4 (several kinds)
Opcode	1 to 3
ModR/M byte (selects registers / addressing form)	0 or 1
SIB byte (scale, index, base)	0 or 1
Displacement	0, 1, 2 or 4
Immediate value	0, 1, 2 or 4
Total	1 to 15

Table 1: Optional fields in an x86 instruction. The decoder cannot know where the next instruction starts until it has interpreted the current one.

Where CISC is used. x86 dominates desktops, laptops and servers because of its enormous software base. Modern x86 chips are very fast, but they spend a large amount of silicon on decoding, caching and out-of-order machinery, which is the opposite of what a coin-cell device wants.

6. Answers to the Case Study Questions

Question 1: What is RISC? What is CISC?

RISC stands for Reduced Instruction Set Computer. It is a design style in which the processor offers a small, regular set of simple instructions, most of which complete in about one clock cycle. Memory is accessed only by load and store instructions, and everything else is done on registers. ARM, RISC-V and MIPS follow this style.

CISC stands for Complex Instruction Set Computer. Here the processor offers a large set of instructions, including powerful ones that read memory, compute and write a result in one step. Instructions vary in length and in the number of cycles they need. x86, VAX and the Motorola 68000 are examples.

The one-line difference is where the effort goes: RISC moves effort into the compiler and keeps hardware simple, while CISC moves effort into the hardware so that programs can be shorter.

Question 2: Why do many RISC instruction sets use fixed-length instructions?

A fixed length makes the next instruction's address trivially known. If every instruction is 4 bytes, the program counter simply adds 4. The fetch unit can therefore keep reading ahead without waiting for the decoder, and several instructions can sit in the pipeline at once. Because the fields are in the same bit positions every time, decoding is a matter of wiring, not of logic that must examine the instruction first. Fixed width also fits neatly into memory words and cache lines.

The disadvantage is code size. A simple operation such as incrementing a register still occupies the full width. Designers of embedded RISC chips therefore added short 16-bit encodings, such as ARM Thumb-2 and the RISC-V compressed extension. These mix two sizes, but the sizes are limited and easy to tell apart from the first bits, so the decoder stays simple.

Question 3: How does instruction complexity influence instruction decoding?

Decoding complexity grows with the number of formats the decoder must recognise. With a handful of fixed formats, a RISC decoder is a small block of combinational logic. With x86, the decoder must scan prefixes, work out the opcode length, check whether a ModR/M or SIB byte follows, read the right number of displacement and immediate bytes, and only then know the instruction's length. Some instructions are decoded through a microcode ROM. Modern x86 processors use several decoders working in parallel and translate each instruction into simpler micro-operations, which is effective but costs silicon area and energy. In low-power designs this overhead is the main reason to avoid complex decoding.

Question 4: Compare the typical addressing-mode approaches in RISC and CISC.

RISC machines restrict addressing to a few simple forms. The usual set is register, immediate (a constant inside the instruction) and base register plus a small offset. Only load and store instructions use memory addresses, so every arithmetic instruction is guaranteed to run without waiting for memory.

CISC machines provide many forms. x86, for instance, can compute an address as base + index x scale + displacement, and arithmetic instructions can take one operand directly from memory. Older CISC designs, such as the VAX, even allowed memory-to-memory operations and multiple levels of indirection.

Aspect	RISC	CISC
Number of modes	Few (about three)	Many (often ten or more)
Memory operands in arithmetic	Not allowed	Allowed
Address calculation	Simple add	Multi-term, possibly with shifts
Effect on hardware	Easy to pipeline	Complex, variable timing
Effect on program	More instructions	Shorter code

Some modern RISC chips, ARM in particular, add conveniences such as auto-increment, but the range of modes is still far narrower than in x86.

Question 5: Can a CISC processor outperform a RISC processor? Explain with one reason.

Yes. The reason is **code density**. A single CISC instruction can stand in for several RISC instructions, so the same program occupies fewer bytes. Fewer bytes mean less instruction-fetch traffic and a better chance that the hot loop fits in a small instruction cache. When memory bandwidth or cache size is the bottleneck, this can outweigh the higher cost of decoding each instruction. A concrete case is a block-copy loop: on x86 it is one instruction, while on a RISC machine it is a loop that must be fetched on every pass.

It must be added that performance is a property of the whole implementation and not only of the instruction set. A well-built RISC chip can beat a poorly built CISC chip, and the reverse is also true.

7. Assignment (a): Comparison Table

The table below compares the two styles across twelve parameters. The assignment asked for at least five.

Parameter	RISC	CISC
Instruction set size	Small and simple	Large, including complex instructions
Instruction length	Mostly fixed (for example 32 bits)	Variable (1 to 15 bytes in x86)
Cycles per instruction	Usually about 1 in a pipeline	Varies, often several
Memory access	Only through LOAD / STORE	Most instructions can use memory operands
Addressing modes	Few: register, immediate, base + offset	Many and complex
Register file	Large (for example 32 general-purpose registers)	Smaller (for example 8 to 16)
Decoder / control unit	Simple, usually hardwired	Complex, often microprogrammed
Code size	Larger	Smaller (higher code density)
Pipelining	Easy and efficient	Harder because instructions are uneven
Power and chip area	Lower	Higher
Role of the compiler	Does more of the optimisation	Hardware does more per instruction
Typical examples	ARM, RISC-V, MIPS	Intel x86, VAX, Motorola 68000

Table 2: RISC versus CISC across twelve parameters.

8. Assignment (b): Example Instruction Sequences

Example 1: counter = counter + 1

The variable `counter` lives in memory and must be increased by one.

CISC (x86, Intel syntax):

```
ADD  DWORD PTR [counter], 1      ; read memory, add 1, write back
```

RISC (ARM): R0 holds the address of the variable.

```
LDR  R1, [R0]        ; load the value from memory
ADD  R1, R1, #1      ; add 1 inside the register
STR  R1, [R0]        ; write the result back
```

Analysis. The x86 version is one instruction, around 7 bytes in 32-bit mode (opcode, addressing byte, a 4-byte address and a 1-byte constant). The 32-bit ARM version uses three instructions of 4 bytes each, so 12 bytes. With the compact Thumb-2 encoding the same three steps take roughly 6 bytes, which shows how much of the density gap the compressed RISC encodings close. Internally, a modern x86 core splits its single instruction into the same load, add and store steps, so the hardware work is similar. The difference is that RISC exposes these steps to the compiler, while x86 hides them behind a decoder.

Example 2: Copying a block of N bytes

CISC (x86): with ESI = source, EDI = destination and ECX = N.

```
REP MOVSB            ; repeat: copy a byte, advance both pointers, ECX = ECX - 1
```

RISC (ARM): with R0 = source, R1 = destination and R3 = N.

```
loop: LDRB R2, [R0], #1   ; load a byte, then move the source pointer
      STRB R2, [R1], #1   ; store the byte, then move the destination pointer
      SUBS R3, R3, #1     ; decrement the count and set flags
      BNE  loop           ; repeat until the count reaches zero
```

Analysis. The x86 sequence is a single instruction, so it is tiny in code size and a modern CPU can run it with specially optimised microcode. The ARM loop needs four instructions per byte copied, and each of them has to be fetched, decoded and executed on every pass. In return, every one of the four steps is

simple, has a fixed length and takes a known time, which is helpful when a deadline has to be proved. For long copies RISC chips usually rely on wider loads, unrolled loops or a DMA controller, which an embedded device normally has.

Bonus example 3: a = b + c with all three variables in memory

```
; x86                          ; ARM
MOV  EAX, [b]                  LDR  R1, [R4]     ; b
ADD  EAX, [c]                  LDR  R2, [R5]     ; c
MOV  [a], EAX                  ADD  R3, R1, R2
                               STR  R3, [R6]     ; a
```

Here the x86 code is three instructions and the ARM code is four. The extra instruction is the price of the load/store rule, and in return the ARM ADD works purely on registers and can never stall waiting for memory.

What the examples show

- CISC gives shorter programs and does more per instruction.
- RISC gives longer but regular programs whose timing is easy to predict.
- Newer RISC encodings (Thumb-2, RISC-V compressed) reduce the size difference.

9. Pipelining and Why It Favours RISC

A pipeline overlaps the execution of several instructions, in the same way as an assembly line. A classic RISC pipeline has five stages: instruction fetch (IF), decode and register read (ID), execute (EX), memory access (MEM) and write back (WB). When each stage takes one cycle and every instruction uses the same stages, a new instruction can start every cycle:

Instruction	1	2	3	4	5	6	7
I1	IF	ID	EX	MEM	WB		
I2		IF	ID	EX	MEM	WB	
I3			IF	ID	EX	MEM	WB

Figure 2: Three instructions overlapping in a five-stage pipeline. From cycle 5 onward one instruction finishes per cycle.

Pipelining works best when instructions are uniform. Fixed length means the fetch stage never has to guess. A single memory access stage, used only by loads and stores, means no instruction needs memory twice. Simple, short-running instructions mean the stages take about the same time. CISC instructions break these conditions: some need several memory accesses, some run for a variable number of cycles, and some are so long that the pipeline has to stall. High-performance x86 chips overcome this by translating into micro-operations first, so they pipeline the simple internal operations instead. The extra translation stage is a cost that RISC simply does not have.

Pipeline hazards (data, control and structural) exist in both styles. RISC's regular structure makes them easier to detect and handle with small, cheap logic such as forwarding paths and simple branch handling, which suits a small chip.

10. Power and Energy: The Core of the Embedded Decision

The dynamic power of a digital circuit is commonly estimated as:

```
P(dynamic)  ~  a  x  C  x  V^2  x  f
```

where **a** is the fraction of gates that switch in a cycle, **C** is the total capacitance (roughly, how much logic there is), **V** is the supply voltage and **f** is the clock frequency. Several of these terms are influenced by the ISA.

- **Less logic (smaller C).** A simple fixed-format decoder and no microcode ROM mean fewer gates to power.
- **Less switching (smaller a).** A decoder that only slices bits does less work per instruction than one that interprets many optional fields.
- **Lower frequency for the same work (smaller f).** When instructions are efficient and the pipeline stays full, the job can finish at a modest clock, and lower clocks allow a lower voltage, which helps because voltage appears squared.
- **Energy = power x time.** A task that finishes quickly and lets the chip return to sleep spends less total energy, so sleep modes matter as much as active power.

Many embedded devices are idle most of the time, so leakage current and wake-up cost are also important. Microcontroller families built on RISC cores generally pay close attention to this, offering several sleep states and fast wake-up. The ISA does not guarantee low power by itself, because implementation and manufacturing process matter too, but a simple ISA makes a small, efficient implementation much easier.

11. Assignment (c): Processor Recommendation

Recommendation: use a RISC-style processor, for example an ARM Cortex-M class microcontroller or a RISC-V microcontroller. The reasoning follows.

1. Lower power consumption

As Section 10 explained, a simple decoder with fixed-length instructions means less logic and less switching. For a device that must run from a battery for a long time, this is the strongest single reason.

2. Smaller and cheaper silicon

Less control logic and no large microcode ROM allow a smaller die. A smaller die costs less, produces less heat and leaves room for useful peripherals on the same chip, such as an ADC, timers and radio interfaces.

3. Predictable timing

Many embedded jobs, such as reading a sensor at an exact interval or driving a motor, are real-time. Instructions that mostly take one cycle make the worst-case execution time easier to calculate and to prove.

4. Efficient use of a pipeline at a low clock

Uniform instructions let a short pipeline stay full. Useful work therefore gets done without a high clock rate, and a lower clock saves further power.

5. A mature ecosystem

ARM and RISC-V both have solid compilers, debuggers, libraries and real-time operating systems. RISC-V, being an open ISA, also avoids licence fees for the ISA itself, which can help with cost on large production runs.

Decision matrix

To make the reasoning visible, the table below scores each criterion from 1 (poor) to 5 (excellent). The weights and scores are my own judgement, not measurements, and they are meant to show how the decision is built rather than to give exact numbers.

Criterion	Weight	RISC	CISC	Weighted RISC	Weighted CISC
Energy per task	5	5	2	25	10
Chip area and cost	4	5	2	20	8
Timing predictability	4	5	2	20	8
Code density	2	3	5	6	10
Software legacy / PC compatibility	1	2	5	2	5
Total				73	41

Table 3: Illustrative weighted decision matrix for a battery-powered embedded device.

Risks and how to reduce them

- **Larger code.** Use a compressed instruction set (Thumb-2 or RISC-V C extension) and compiler size optimisation. Flash memory is inexpensive.
- **Not enough performance for one heavy task.** Use a hardware accelerator or DMA for that task, a common approach in microcontrollers.
- **Existing x86-only software.** If the product had to run legacy x86 software unchanged, a CISC processor would be justified. That is not the case for a new, dedicated embedded device.

When CISC would be the right call. If the device were a mains-powered industrial PC running standard desktop software, then x86's software compatibility and performance would outweigh power savings. The recommendation therefore depends on the stated use case, which is exactly what the problem statement asked for.

12. How Blurred the Line Is Today

The strict RISC versus CISC contrast was clearest in the 1980s. Today the two groups have borrowed from each other. x86 processors decode into RISC-like micro-operations and run them on a deeply pipelined, out-of-order core. ARM, the best-known RISC family, now includes instructions that do considerably more than a 1980s RISC instruction would, such as multiple-register loads and stores, vector (SIMD) operations and cryptographic instructions. Even so, the distinction still shows up where it counts for this case study: in how complicated the decoder is, how regular the instruction format is, and how much silicon and energy go into the front end of the processor. For small, low-power devices these differences remain large.

13. Conclusion

CISC tries to get more done in each instruction and so produces compact programs. RISC tries to make each instruction as cheap and quick as possible and so produces simple, efficient hardware. CISC can outperform RISC in some situations, mainly because of code density, and modern processors have borrowed ideas from both sides. For a battery-powered embedded device, however, the priorities are low energy per task, small and cheap silicon, and predictable timing. A RISC processor serves all three, and its main drawback, larger code, is largely repaired by compressed instruction sets. For that reason, the recommended choice is a RISC-based processor such as an ARM Cortex-M or a RISC-V microcontroller.

14. Glossary

Term	Meaning
ISA	Instruction set architecture: the instructions, registers and encoding that software sees.
CPI	Cycles per instruction: the average number of clock cycles one instruction takes.
Pipeline	A technique that overlaps the stages of several instructions to raise throughput.
Microcode	A small internal program that implements a complex instruction as simpler steps.
Micro-operation	A simple internal operation that a complex instruction is broken into.
Load/store architecture	A design in which only load and store instructions access memory.
Addressing mode	The rule that tells an instruction how to find its operand.
Code density	How much work a program does per byte of code.
Thumb-2	ARM's mixed 16-bit and 32-bit instruction encoding that reduces code size.
Hardwired control	A control unit built from logic gates instead of microcode.

15. References

Only list the sources you have actually read or used, and check edition details before submitting.

D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface* (RISC-V Edition), Morgan Kaufmann.

W. Stallings, *Computer Organization and Architecture: Designing for Performance*, Pearson.

J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, Morgan Kaufmann.

D. A. Patterson and D. R. Ditzel, "The Case for the Reduced Instruction Set Computer," *ACM SIGARCH Computer Architecture News*, 1980.

Intel 64 and IA-32 Architectures Software Developer's Manual, Intel Corporation.

Arm Architecture Reference Manual, Arm Ltd.

The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, RISC-V International.